23CSE211 - Design and Analysis of Algorithms

Attempts on solving Flow-Free - 1

*Graphs, BFS, Sorting, A**

Submitted by:

Akilan S S	–	CB.SC.U4CSE24707
Govind Nair	–	CB.SC.U4CSE24720
M V Sai Kartik	–	CB.SC.U4CSE24744
Sriram Tatikonda	–	CB.SC.U4CSE24753

Submitted to:

Dr. Vidhya Balasubramanian

Wednesday 17th December, 2025

Contents

1 Introduction

Flow-free is a grid based game, where in an n by n grid, there are k pairs of colours (or terminals), which has to be connected by drawing pipes on the grid. No two pipes must intersect, and they should fill the grid as much as possible. In this project, we attempt to solve the game Flow-Free with traditional algorithm techniques, presented till date in 23CSE211, in a collaborative manner through the following strategies.

1. Naive BFS
2. Sorted Euclidean Distances
3. A* Heuristics

2 Naive BFS

2.1 Description

Algorithm first uses helper function to get all the available terminals (terminals that are not connected). Then the helper function uses DFS to check which which terminals can be connected and then returns the terminals as array. The we use a helper function to get a random index and choice one terminal. Then we run BFS on the terminal one end until we reach the other end. A parent matrix is kept that keeps track of the parent of each node while running BFS, then we use this parent matrix to backtrack the path once we reach the other terminal end.

2.2 Time Complexity Analysis

Let n be the number of rows (or columns) in the grid, and k be the number of available terminals.

- **Scan for terminals:** $O(n^2)$
  Time required to scan the board and identify available terminals.
- **DFS Validity Check:** $O(k \cdot n^2)$
  Running DFS on all available terminals to verify if a path exists. In the worst case, each DFS traversal visits every cell in the grid (n^2).
- **Main BFS Execution:** $O(n^2)$
  Time required to run BFS from the chosen start terminal to the target. In the worst case, BFS visits every vertex and edge once.

Total Time Complexity: $O(kn^2)$

Algorithm 1 Breadth-First Search for Terminal Connection

Require: Board object
Ensure: Path (list of (row, col) pairs)

$path \leftarrow$ empty list
$terminals \leftarrow \text{GetTerminals}(board)$
if $terminals$ is empty **then**
    return $path$
end if
▷ Choose a random terminal to attempt connection
$terminal \leftarrow$ random choice from $terminals$
$color \leftarrow terminal.color$
$(start\_row, start\_col) \leftarrow (terminal.row, terminal.col)$
▷ BFS Initialization
$queue \leftarrow$ empty queue
$visited[N][N] \leftarrow$ all false
$parents[N][N] \leftarrow$ initialize to $(-1, -1)$
$visited[start\_row][start\_col] \leftarrow$ **true**
$queue.\text{Push}((start\_row, start\_col))$
$found \leftarrow$ **false**
$(end\_row, end\_col) \leftarrow (-1, -1)$
while $queue$ is not empty **do**
    $(row, col) \leftarrow queue.\text{Pop}()$
    ▷ Check if we reached the matching terminal
    if $board[row][col]$ is terminal **and** $(row, col) \neq (start\_row, start\_col)$ **then**
        $found \leftarrow$ **true**
        $(end\_row, end\_col) \leftarrow (row, col)$
        break
    end if
    $neighbors \leftarrow \text{GetNeighbors}(board, row, col, color)$
    for all (nr, nc) in $neighbors$ **do**
        if $visited[nr][nc] =$ **false then**
            $visited[nr][nc] \leftarrow$ **true**
            $parents[nr][nc] \leftarrow (row, col)$
            $queue.\text{Push}((nr, nc))$
        end if
    end for
end while
if $found =$ **false then**
    return $path$
end if
▷ Reconstruct path from End $\rightarrow$ Start
$curr \leftarrow (end\_row, end\_col)$
while $curr \neq (-1, -1)$ **do**
    $path.\text{Append}(curr)$
    $curr \leftarrow parents[curr.row][curr.col]$
end while
$\text{Reverse}(path)$
return $path$

3 Sorted Euclidean Distances

3.1 Description

In this strategy, we try to connect the paths whose terminals are closer. This allows us to ensure that no pipes are drawn conflicting an obvious path. In this strategy, we first compute the euclidean distances between the terminal. We then sort the distances, and connect the terminals from the shortest distance to the longest distance. Here, we use a variant of Shell Sort with Hibbard Sequence.

3.2 Time Complexity Analysis

- **Shell Sort:** $O(n\sqrt{n})$
  We use hibbard sequence to perform insertion sort
- **BFS Execution:** $O(n^2)$
  Time required to run BFS from the chosen start terminal to the target. In the worst case, BFS visits every vertex and edge once.

Algorithm 2 Path selection algorithm using shell sort

```
1: function ALGORITHM(board)
2:     create empty dictionary terminals
3:     for r ← 0 to GRID − 1 do
4:         for c ← 0 to GRID − 1 do
5:             if board[r][c].isTerminal then
6:                 append (r, c) to terminals[board[r][c].color]
7:             end if
8:         end for
9:     end for
10:    create empty list pairs
11:    for all (color, T) in terminals do
12:        if |T| = 2 then
13:            A ← T[0], B ← T[1]
14:            dist ← DISTANCEEUCLID(A, B)
15:            append (color, A, B, dist) to pairs
16:        end if
17:    end for
18:    sort pairs by increasing dist
19:    for all pair in pairs do
20:        if ISALREADYSOLVED(board, pair.A, pair.B) then
21:            remove pair from pairs
22:        end if
23:    end for
24:    for all pair in pairs do
25:        path ← BFSPATH(board, pair.A, pair.B)
26:        if path is not empty then
27:            return path
28:        end if
29:    end for
30:    return empty path
31: end function
```

4 A* Heuristics

4.1 Description

The algorithm solves a Flow Free–style puzzle using a heuristic-driven A* approach. First, all available terminal pairs that can still be connected are identified. These terminals are grouped by color, and for each color with exactly two terminals, a difficulty value is computed based on Manhattan distance and surrounding free space. The terminal pairs are then sorted so that easier connections are attempted first. For each terminal pair, the algorithm runs A* search twice (start $\rightarrow$ end and end $\rightarrow$ start). A* uses Manhattan distance as the heuristic and is enhanced with domain-specific biasing:

- Penalizes creation of 1x1 holes
- Penalizes dead regions without terminals
- Encourages straight paths
- Slightly prefers boundary movement

A parent matrix is maintained during the search to reconstruct the path once the goal is reached. As soon as a valid path is found for any terminal pair, the algorithm returns that path.

4.2 Time Complexity Analysis

Let n be the number of rows or columns in the grid and k be the number of terminal pairs

- **Finding reachable terminals:**
  $O(n^2)$ (grid traversal + BFS checks)
- **Difficulty computation for all pairs:**
  $O(k)$
- **Sorting terminal pairs:**
  $O(k \log k)$
- **A* search (per pair):**
  Nodes explored $\leq n^2$
  Priority queue operations $\rightarrow \log(n^2)$
  Time per A*: $O(n^2 \log n)$
- **Worst case (2 A* runs per pair):**
  $O(k \cdot n^2 \log n)$
- **Total Time Complexity**
  $O(k \cdot n^2 \log n)$

Algorithm 3 FLOW_ASTAR(board)

Require: Board object
Ensure: Path (list of (row, col) pairs representing a valid pipe)

$path \leftarrow$ empty list
▷ Step 1: Get all reachable terminal pairs
$terminals \leftarrow getTerminals(board)$
if $terminals$ is empty **then**
    return empty path
end if
▷ Step 2: Group terminals by color
$groups \leftarrow$ map from color $\rightarrow$ list of terminal cells
for all terminal t in $terminals$ **do**
    $groups[t.color].append((t.row, t.col))$
end for
▷ Step 3: Build candidate terminal pairs with difficulty
$candidates \leftarrow$ empty list
for all color in $groups$ **do**
    if $groups[color].size = 2$ **then**
        $a \leftarrow$ first terminal
        $b \leftarrow$ second terminal
        $difficulty \leftarrow computeDifficulty(board, a, b)$
        $candidates.append((color, a, b, difficulty))$
    end if
end for
if $candidates$ is empty **then**
    return empty path
end if
▷ Step 4: Sort pairs by increasing difficulty
sort $candidates$ by difficulty (ascending)
▷ Step 5: Try A* on each candidate pair
for all candidate in $candidates$ **do**
    $start \leftarrow candidate.a$
    $goal \leftarrow candidate.b$
    $path1 \leftarrow A\_STAR(board, start, goal)$
    $path2 \leftarrow A\_STAR(board, goal, start)$
▷ Choose the shorter valid path
    if $path1$ not empty **AND** $path2$ not empty **then**
        $chosen \leftarrow shorter(path1, path2)$
    else if $path1$ not empty **then**
        $chosen \leftarrow path1$
    else if $path2$ not empty **then**
        $chosen \leftarrow path2$
    else
        CONTINUE
    end if
    if $chosen$ not empty **then**
        return $chosen$
    end if
end for
return empty path

Algorithm 4 A_STAR(board, start, goal)

Require: Board, start cell, goal cell
Ensure: Shortest valid path between start and goal
$priorityQueue \leftarrow$ empty min-heap (ordered by f-score)
$bestG[GRID][GRID] \leftarrow$ infinity
$parent[GRID][GRID] \leftarrow (-1, -1)$
$g(start) \leftarrow 0$
$h(start) \leftarrow ManhattanDistance(start, goal)$
$f(start) \leftarrow g + h$
push $start$ into $priorityQueue$
$bestG[start] \leftarrow 0$
while $priorityQueue$ is not empty **do**
    $current \leftarrow$ pop node with smallest f-score
    if $current = goal$ **then**
        return $reconstructPath(parent, goal)$
    end if
    for all neighbor of $current$ (up, down, left, right) **do**
        if neighbor is outside grid **then**
            CONTINUE
        end if
        if neighbor has pipe **AND** neighbor $\neq goal$ **then**
            CONTINUE
        end if
        if neighbor is terminal of different color **AND** neighbor $\neq goal$ **then**
            CONTINUE
        end if
        $newG \leftarrow g(current) + 1$
        $newH \leftarrow ManhattanDistance(neighbor, goal)$
        $bias \leftarrow 0$
        if placing pipe creates 1x1 hole **then**
            $bias \leftarrow bias + holePenalty$
        end if
        if placing pipe creates dead space **then**
            $bias \leftarrow bias + deadSpacePenalty$
        end if
        if direction same as parent direction **then**
            $bias \leftarrow bias - straightBonus$
        end if
        if neighbor lies on grid boundary **then**
            $bias \leftarrow bias - boundaryBonus$
        end if
        $newF \leftarrow newG + newH + bias$
        if $newG < bestG[neighbor]$ **then**
            $bestG[neighbor] \leftarrow newG$
            $parent[neighbor] \leftarrow current$
            push $neighbor$ with $f = newF$ into $priorityQueue$
        end if
    end for
end while
return empty path

5 Test System

We believe the philosophy behind Test Driven Development (in the context of algorithms), and we made sure we get this right at the first phase. We have developed a comprehensive

system to generate and review algorithms to benchmark them next to each other. We believe this will provide useful insights in analysing algorithms, especially those which are heuristic in nature.

5.1 Fast Test Suite Generator

A super fast test-suite generator was developed in Python to develop testcases for different grid sizes and terminal pairings. Every test case was exported as a CSV for compatibility with the raylib project.

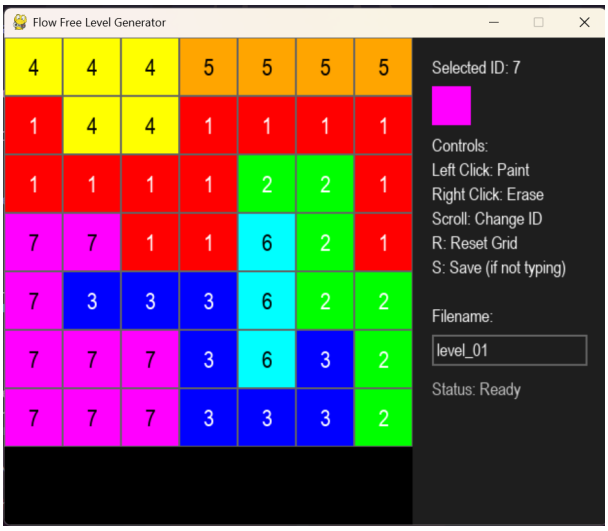


Figure 1: Test-Suite Generator

5.2 Referee System

Another script was made to automate the task of going through different algorithms for a particular test case. For every level loaded to review, the algorithms load sequentially for human scoring. We are planning to automate the process of assigning scores in the future.

5.3 Test Cases

Algorithms developed in the course of this phase, eerily seem like they could indeed solve the "NP-hard" problem. To humble ourselves, (or maybe Akilan doesn't like the idea that P could be NP, which he believes it would lead to the collapse of the universe), we have currently hand crafted certain test cases based on different possible points of failure.

1. Terminals which are closer together, but takes a longer path as it's correct solution
2. Paths which are "bounded" by each other

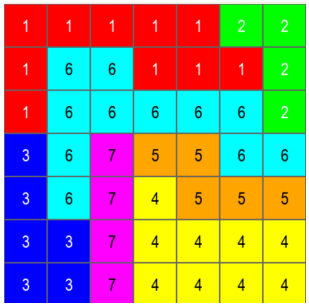
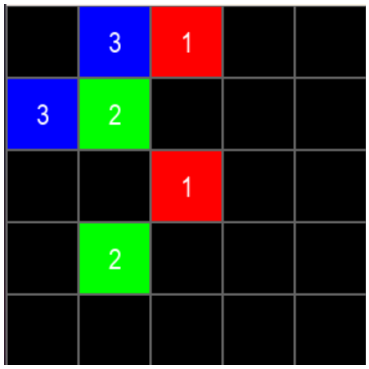
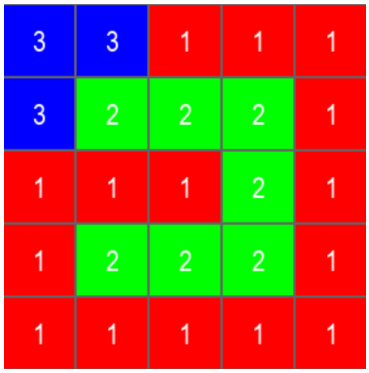


Figure 2: This test case requires alot of turns to fill the board



(a) Terminals close to each other



(b) Optimal Solution for 3 (a)

Figure 3: A level where the shortest path strategy might fail

In a similar fashion, including trivial test cases, around 10 test cases for N = 5 and N = 7 were developed. The review system which analysed the algorithms, showed not much of a difference on average, classifying them in the same category of solvers. In the upcoming phases, we hope we make better and better solvers.

6 Conclusion and Future Directions

In this project, different strategies were explored and a comprehensive testing system was developed to analyse different strategies throughout the project. For the next phase of this project, we look forward in applying different problem solving paradigms, such as greedy, dynamic programming, network flow and heuristics.

7 LLM Disclosure

Here is a series of prompts (or the kind of prompts) given to LLMs, to help us assist during the course of this project.

1. Makeflie fixes (with code snippets)
   This prompt was given to generate the correct makefile which aligns with the user's C++ compiler and Raylib location.

2. UI offset fixes (with code snippet)
   This prmopt was given to fix an alignment issue with the game window and the grid. It resolved the issue where the padding and grid offset parameters were used interchangedly.

3. LaTeX pseudocode formatting (pseudocode given)
   This prompt was given to convert the pseudocode entered by the user to make it LateX-style using the algpseudocode and algorithmic packages.

4. Can we not check if a terminal isreachable to its destination terminal using A* itself, instead of BFS?
   Prompted this because, I wanted to know if its possible to solve the grid purely with A* and not use any other algorithm

5. I have this idea of using biases, which will act as weights and will direct the AI to pick the best possible move while finding the A* path
   Prompted this to find if we can use wacky weights approach in A* algorithm

6. What is the worst-case time complexity of a single A call in this grid?
   Had a doubt if its $O(N^2 \log N^2)$ or not and if I understood the priority queue behavior